WEEK-6
Collective Communications and Error Handling in MPI

Niharika Gawade
240905284
Roll no:30
CSE-D

1) Implement a MPI program using N processes to find 1! + 2! +.....+N!. Use scan. Also, handle different errors using error handling routines.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
   int rank, size, i, fact = 1, sum, total;
   int err;

   err = MPI_Init(&argc, &argv);
   if (err != MPI_SUCCESS)
   {
      printf("MPI_Init failed\n");
      return 1;
   }

   err = MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   if (err != MPI_SUCCESS)
   {
      printf("MPI_Comm_rank failed\n");
      MPI_Abort(MPI_COMM_WORLD, 1);
   }

   err = MPI_Comm_size(MPI_COMM_WORLD, &size);
   if (err != MPI_SUCCESS)
   {
      printf("MPI_Comm_size failed\n");
      MPI_Abort(MPI_COMM_WORLD, 1);
   }

   if (rank == 0)
      printf("Number of processes = %d\n", size);

   i = rank + 1;

   for (int j = 1; j <= i; j++)
      fact *= j;

   err = MPI_Scan(&fact, &sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
   if (err != MPI_SUCCESS)
   {
```

```
        printf("MPI_Scan failed\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    if (rank == size - 1)
        total = sum;

    MPI_Bcast(&total, 1, MPI_INT, size - 1, MPI_COMM_WORLD);

    if (rank == 0)
        printf("Sum = %d\n", total);

    MPI_Finalize();
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpicc p1.c -o p1 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpirun -np 4 ./p1
Number of processes = 4
Sum = 33
```

2) Implement a MPI program to read a 3 X 3 matrix. Enter an element to be searched in the root process. Find the number of occurrences of this element in the matrix using three processes.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, matrix[3][3], row[3], key, count, total;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0)
    {
        printf("Enter 3 X 3 matrix:\n");

        for (int i = 0; i < 3; i++)
            for (int j = 0; j < 3; j++)
                scanf("%d", &matrix[i][j]);

        printf("Enter element to search: ");
        scanf("%d", &key);
    }

    MPI_Bcast(&key, 1, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Scatter(matrix, 3, MPI_INT, row, 3, MPI_INT, 0, MPI_COMM_WORLD);
```

```
    count = 0;

    for (int i = 0; i < 3; i++)
    {
        if (row[i] == key)
            count++;
    }

    MPI_Reduce(&count, &total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0)
        printf("Number of occurrences = %d\n", total);

    MPI_Finalize();
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpicc p2.c -o p2 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpirun -np 4 ./p2
Enter 3 X 3 matrix:
1 4 5
9 0 1
5 4 1
Enter element to search: 1
Number of occurrences = 3
```

3) Implement a MPI program to read 4 X 4 matrix and display the following output using four processes.

I/p matrix:
1 2 3 4
1 2 3 1
1 1 1 1
2 1 2 1
O/p matrix:
1 2 3 4
2 4 6 5
3 5 7 6
5 6 9 7

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;
    int a[4][4];
    int row[4];
```

```
int result[4][4];

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (size != 4)
{
   if (rank == 0)
      printf("Run with 4 processes\n");

   MPI_Finalize();
   return 1;
}

if (rank == 0)
{
   printf("Enter 4 X 4 matrix:\n");

   for (int i = 0; i < 4; i++)
      for (int j = 0; j < 4; j++)
         scanf("%d", &a[i][j]);
}

MPI_Scatter(a, 4, MPI_INT, row, 4, MPI_INT,
          0, MPI_COMM_WORLD);

MPI_Scan(row, result[rank], 4, MPI_INT,
        MPI_SUM, MPI_COMM_WORLD);

if (rank == 0)
{
   MPI_Gather(MPI_IN_PLACE, 0, MPI_INT,
            result, 4, MPI_INT, 0, MPI_COMM_WORLD);
}
else
{
   MPI_Gather(result[rank], 4, MPI_INT,
            result, 4, MPI_INT, 0, MPI_COMM_WORLD);
}

if (rank == 0)
{
   printf("O/p matrix:\n");

   for (int i = 0; i < 4; i++)
   {
      for (int j = 0; j < 4; j++)
         printf("%d ", result[i][j]);

      printf("\n");
```

```
    }
  }

  MPI_Finalize();

  return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpicc p3.c -o p3 -lm
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpirun -np 4 ./p3
Enter 4 X 4 matrix:
1 2 3 4
1 2 3 1
1 1 1 1
2 1 2 1
O/p matrix:
1 2 3 4
2 4 6 5
3 5 7 6
5 6 9 7
```

4) Implement parallel odd even transposition sort.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
  int rank, size;
  int local, received;
  int a[100], result[100];

  MPI_Init(&argc, &argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);
  MPI_Comm_size(MPI_COMM_WORLD, &size);

  if (rank == 0)
  {
    printf("Enter %d elements:\n", size);

    for (int i = 0; i < size; i++)
      scanf("%d", &a[i]);
  }

  MPI_Scatter(a, 1, MPI_INT, &local, 1, MPI_INT, 0, MPI_COMM_WORLD);

  for (int phase = 0; phase < size; phase++)
  {
```

```
    if (phase % 2 == 0)
    {
        if (rank % 2 == 0 && rank + 1 < size)
        {
            MPI_Sendrecv(&local, 1, MPI_INT, rank + 1, 0,
                         &received, 1, MPI_INT, rank + 1, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            if (local > received)
                local = received;
        }
        else if (rank % 2 == 1)
        {
            MPI_Sendrecv(&local, 1, MPI_INT, rank - 1, 0,
                         &received, 1, MPI_INT, rank - 1, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            if (local < received)
                local = received;
        }
    }
    else
    {
        if (rank % 2 == 1 && rank + 1 < size)
        {
            MPI_Sendrecv(&local, 1, MPI_INT, rank + 1, 0,
                         &received, 1, MPI_INT, rank + 1, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            if (local > received)
                local = received;
        }
        else if (rank % 2 == 0 && rank > 0)
        {
            MPI_Sendrecv(&local, 1, MPI_INT, rank - 1, 0,
                         &received, 1, MPI_INT, rank - 1, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);

            if (local < received)
                local = received;
        }
    }
}

MPI_Gather(&local, 1, MPI_INT, result, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (rank == 0)
{
    printf("Sorted array:\n");

    for (int i = 0; i < size; i++)
        printf("%d ", result[i]);
```

```
        printf("\n");
    }

    MPI_Finalize();
    return 0;
}
```

OUTPUT

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_6$ mpirun -np 5 ./p4
Enter 5 elements:
2 8 9 3 0
Sorted array:
0 2 3 8 9
```